USC Viterbi
School of Engineering

EE 599: Designing and Building Autonomous AI Agents

Units: 2
Term: Fall 2026
Instructor: Arash Saifhashemi
Meeting time, location, office hours, TA, and contact details: TBD

Course description

This course introduces the design and implementation of AI agents: software systems in which a large language model can choose actions, use external tools, and take multiple steps to complete a task.

Students first build a simple agent loop in Python and then use selected libraries to study task sequencing, tool use, saved state, retrieval (RAG), multimodal inputs, evaluation, human approval, and security.

Across the course, three ideas recur: how work is coordinated, how information is stored, and how a system can improve after examining previous results.

Course scope and workload

Expect about four hours of work outside class each week, in addition to the weekly meeting.

Students complete programming assignments and one semester project developed in stages. In-class exercises support this work and are not separate graded assignments.

The course uses the OpenAI Agents SDK and LangGraph for different examples, but students are assessed on the underlying concepts rather than on memorizing either API. Training model weights, reinforcement learning, genetic programming, and distributed agent systems may be discussed briefly but are not required implementations.

Learning objectives

By the end of the course, students will be able to:

1. **LLM and agent foundations:** Explain at a high level how an LLM generates output from its input and why its answers can vary or be confidently incorrect; explain what this course means by an agent and by reasoning.
2. **Agent loop:** Build a loop that sends a task to a model, executes any requested tools, returns tool results to the model, and stops safely, using structured inputs, validation, and failure handling.
3. **Orchestration choices:** Choose among fixed workflows, single-agent loops, routing, handoffs, and a manager agent that coordinates specialist agents based on task needs.
4. **MCP and interoperability:** Explain MCP's purpose and host, client, and server architecture; distinguish tools, resources, and prompts; and connect an external capability while accounting for permissions and places where data or control passes between systems with different privileges.
5. **Context, state, and memory:** Distinguish information sent to the model for one call, information stored during one run, saved execution state, information selected from external storage, and information retained across separate runs.

6. **Reflection, adaptation, learning, and training:** Distinguish producing a critique, changing a prompt/memory/code/control rule, demonstrating sustained improvement on new tasks, and training model weights.
7. **Evaluation:** Evaluate agent behavior using repeatable tasks, pass/fail checks or scoring rubrics, repeated-trial metrics (`pass@k` for at least one success and `pass^k` for all k attempts succeeding), records of model and tool calls, cost, response time, and analysis of failed cases.
8. **Reliable and responsible agent application:** Build and defend an AI-agent application that records its state, requires human approval before a tool sends a message, spends money, or changes/deletes data, limits permissions, and documents how it recovers from failed or interrupted runs.
9. **Multimodal agents:** Process non-text inputs (such as UI screenshots or charts) using Vision-Language Models to inform agent actions.

Recommended preparation

Students should be comfortable writing and debugging Python, calling HTTP APIs, using Git, and reading basic type annotations. Familiarity with asynchronous programming is helpful but not required.

Prior coursework on transformers or LLMs is not required. The course introduces how an LLM generates text, what information it can use from its current input, why outputs can vary, and why fluent output is not necessarily correct. In this course, **reasoning** means observable multi-step problem solving, such as breaking down a task or using intermediate tool results; the term does not imply human-like understanding or guarantee a correct answer.

Python features beyond the recommended preparation, including Pydantic and `asyncio`, are introduced when an assigned example first needs them. Decorators and other Python features are introduced only when an assigned example uses them.

There are no formal prerequisites.

Software used in the course

Common foundation

- Python 3.11 or newer, Git, `pytest`, and a documented environment setup.
- A GitHub account connected to the student's university email address.
- Visual Studio Code and Codex.

How platforms are used

Role in the course	Technology	Expected depth
Tools, handoffs, and traces	OpenAI Agents SDK	Students practice tool calls, handoffs, and inspecting records of a run; SDK syntax is not assessed.
State, interruption, and recovery	LangGraph	Students implement explicit state, save copies of execution state, pause for human input, and resume work after an interruption.
Interoperability standard	MCP	Students learn the host, client, and server model, tools/resources/prompts, how a client asks a server what it offers, standard input/output (`stdio`) for communication between local processes, Streamable

Role in the course	Technology	Expected depth
		HTTP for remote communication, and how a server can ask the user for missing information or confirmation while a tool call is still running. They then connect one capability and examine its permissions and its trust boundary, meaning the point where data or control passes between two systems that do not have the same permissions.
Vector search / RAG	ChromaDB or FAISS	Students implement simple in-memory vector embeddings to enable their agents to search and retrieve knowledge from external documents.
Optional software	Additional model providers, memory or evaluation libraries, and browser-automation tools	Used only when a project needs them; not required for all students.

You may use other tools for your final project. Check with the instructor first, because starter code and in-class help are built around the tools listed above.

Early examples use plain Python, with no framework, so you can see every step of the loop between the model and its tools before a library hides it.

The course implements one controlled improvement cycle: inspect a trace of a previous run, propose one change, test the change, and either keep it or undo it. Training model weights and reinforcement learning are overview topics only.

Course format

The class meets once a week for 110 minutes, with a short break. Each meeting mixes instruction (lecture, worked examples, and discussion) with in-class programming time. The balance between lecture and in-class programming varies by week. Early weeks typically feature 45-60 minutes of instruction, while later weeks (such as Weeks 12-14) shift to shorter 15-minute “mini-lectures” to maximize hands-on lab and project time.

Each week in the schedule below separates the lecture topics from the student activities. Final presentations take the form of prerecorded videos, shown in class on December 4, 2026.

Bring a charged laptop that can run the course environment to every meeting. You will write and run code in most class sessions, and the room may not have enough power outlets for everyone. During programming time, students work on a guided exercise, an assignment, or their semester project. Exercises completed only during class are not graded unless the weekly schedule identifies them as part of an assignment.

One or two meetings may feature an invited speaker whose work matches the scheduled topic. Affected meetings will be announced at least two weeks in advance. If a speaker is not available, the instructor teaches the listed material.

The semester project develops through an idea, a working prototype, and a tested final version of the same system.

Assessments and grading

Assessment	Weight	Evidence of learning
Four programming assignments	40%	Working code, automated tests, a design explanation of no more than one page, and an explanation of failed cases; 10% each
Project, first version	Required, ungraded	First working version, project plan, architecture block diagram, a defined set of test tasks, initial results, and an improvement plan; receives feedback
Project, final version and recorded video presentation	60%	Improved code, test results that another person can reproduce by following the submitted instructions, analysis of security and human approval, final report, and recorded demonstration
Total	**100%**	

In-class exercises, draft plans, and peer feedback are not graded on their own, but you can reuse that work in an assignment or the project. Attendance is not graded.

The individual project has only two required LMS submissions: the first version on **November 8, 2026**, and the final version on **December 12, 2026**. The first version is ungraded and receives feedback. The **December 4, 2026 in-class presentation** is a required recorded video of at most **three minutes**, uploaded to YouTube as **Unlisted**. Add the video link to the project README before class and include it in the final LMS submission; no separate presentation upload to the LMS is required. The presentation contributes to the final-project grade. Progress checks are informal class activities with no separate upload. The project plan is limited to one page; the architecture block diagram, evaluation results, and other supporting materials are provided separately. Update the diagram for the final submission.

Evaluate the first version on at least 10 cases and the final version on at least 20 cases. Use metrics from Session 6, including `pass@k` and `pass^k`, alongside cost, time, and analysis of failures. State your chosen k and number of trials per case, and compare the baseline, first version, and final version under consistent conditions. The project instructions list the required evaluation materials.

Weekly schedule

Each week separates the **Lecture** (what the instructor will teach) from the **Student Activity / Deliverable** (what you will do in class or have due).

Week 1: LLMs, Reasoning Models, and the First Agent Loop

- **Lecture:** How LLMs generate text, the distinction between a single model call and an “agent”, and building a foundational agent loop (model -> tool -> result).
- **Student Activity:** Build a plain-Python agent loop using Pydantic for structured outputs. Brainstorm semester project topics.

Week 2: Frameworks, Tools, and Testing

- **Lecture:** Agent Framework Abstractions: The role of Harnesses and Runners, enforcing Structured Outputs, and unit testing non-deterministic systems.

- **Student Activity:** Rebuild the Week 1 loop using the SDK and write tests for a custom tool. *(In-class work on Assignment 1)*

Week 3: Interoperability with Model Context Protocol (MCP)

- **Lecture:** The MCP standard, host/client/server architecture, tools vs. resources, and identifying trust boundaries.
- **Student Activity:** Connect an external MCP server to your agent, or expose your custom tool as an MCP server. Start the threat model.
- **Deliverable:** Assignment 1 due at the start of the week.

Week 4: Prompting Frameworks and Orchestration

- **Lecture:** Agent reasoning paradigms (Chain-of-Thought, ReAct), breaking tasks into steps, routing, and manager vs. specialist agents.
- **Student Activity:** Rewrite agent system prompts to use ReAct patterns; implement routing or a handoff between two agents. *(In-class work on Assignment 2)*

Week 5: LangGraph Fundamentals and State Management

- **Lecture:** Representing workflows as graphs (nodes and edges), explicit state shared across steps, and basic execution checkpoints.
- **Student Activity:** Build a basic graph with explicit state and a stop condition; connect an MCP tool.
- **Project preparation:** Discuss your subject and scope in class. Include the project plan with the November 8 first version; no separate submission is required now.

Week 6: Evaluation and Human Approval

- **Lecture:** Pass/fail checks, LLM-as-a-judge scoring rubrics, repeated-trial metrics (`pass@k` and `pass^k`), uncertainty in measured success rates, tracing costs, and requiring human approval before an agent acts.
- **Student Activity:** Write test tasks and scoring rubrics for the agent. Draft the evaluation plan for the semester project.
- **Deliverable:** Assignment 2 due at the start of the week.

Week 7: RAG and Research Agents

- **Lecture:** Retrieval-Augmented Generation (RAG), vector embeddings, and checking claims against retrieved sources.
- **Student Activity:** Build a "Search_Knowledge_Base" tool using simple embeddings to allow the agent to answer questions from a provided document. *(In-class work on Assignment 3)*

Week 8: System Design and Measurement

- **Lecture:** Drawing accurate system diagrams, industry benchmarks (SWE-bench, GAIA), and trading accuracy against cost by running multiple attempts.
- **Student Activity:** Generate and verify a diagram of the semester project; conduct a peer review of a partner's diagram.
- **Deliverable:** Assignment 3 due at the start of the week. The first version of the semester project is due **November 8, 2026** and receives feedback without a grade.

Week 9: Memory and Persistence

- **Lecture:** Context vs. State, LangGraph execution checkpoints, and long-term memory (what the system writes, corrects, and forgets).
- **Student Activity:** Add checkpoints to the project, resume a run from a checkpoint, and implement one long-term memory rule. *(In-class work on Assignment 4)*

Week 10: Coding Agents and Learning from Experience

- **Lecture:** Context Injection: Systemic rules vs. User prompts, reflection on execution traces, and dynamically adapting tool selection based on past failures.

- **Student Activity:** Write instruction files for a repository and implement one controlled improvement cycle based on a failed trace.

Week 11: Human Control and Recovery

- **Lecture:** Interrupting Asynchronous Execution: State forking, execution replay, and the architecture of Human-in-the-loop approval points.
- **Student Activity:** Add an approval point before the riskiest tool call in the project. Test interrupting and replaying a run.

Week 12: Multimodal Agents and Context Management

- **Lecture:** Using Vision-Language Models (VLMs) for visual UI navigation, managing token costs, and prompt caching.
- **Student Activity:** Use Playwright for browser automation combined with a VLM to allow the agent to navigate a UI or extract data from screenshots. Profile step-by-step costs.

Week 13: Project Testing and Refinement

- **Lecture:** Classifying failures (prompt vs. tool vs. control logic vs. context) and designing believable before-and-after comparisons.
- **Student Activity:** Rerun evaluation tests, classify failures, fix one major issue, and record before-and-after metrics.
- **Deliverable:** Assignment 4 due at the start of the week.

Week 14: Agent Security and Threat Modeling

- **Lecture:** Prompt injection, unsafe tool use, excessive permissions, and defenses (least privilege, treating output as data).
- **Student Activity:** Exchange projects with a partner and attempt to exploit their agent using a security checklist. Report findings into the threat model.
- **Project preparation:** Revise the threat model and draft the final results; both become part of the graded final project. There is no separate upload.

Recorded video presentations: December 4, 2026

- **Lecture:** A high-level overview of other frameworks (DSPy, AutoGPT, CrewAI) and future directions in agentic AI.
- **Student Activity:** Show recorded presentations covering the user and task, architecture block diagram, application demonstration, evaluation results, costs, failures, limitations, and individual contribution.
- **Deliverable:** Required prerecorded video presentation on **December 4, 2026**, included in the final-project grade: at most **three minutes**, uploaded to YouTube as **Unlisted**, with the link in the project README before class and in the final LMS submission. The video is the presentation; no separate live presentation is required.

Final period

- Submit reproducible code, report, evaluation results, verification and reproducibility note, and individual contribution statement.

Deliverable: Final project due **December 12, 2026**, through the LMS.

Reading strategy

There is no required textbook. Most weeks use the relevant software documentation or example plus, at most, one reading expected to take no more than 45 minutes. Documentation is assigned only when that software appears in class; students are not expected to follow every agent platform.

Suggested references include:

- ReAct: Synergizing Reasoning and Acting in Language Models
- Anthropic: Building Effective Agents

- OpenAI Agents SDK: orchestration
- LangGraph overview and saved-state execution
- LangChain memory concepts
- Model Context Protocol architecture
- MCP 2026-07-28 specification release
- OpenAI: evaluate agent workflows
- OWASP Top 10 for Agentic Applications 2026
- ReasoningBank: learning from experience
- Reflexion: Language Agents with Verbal Reinforcement Learning
- Generative Agents: Interactive Simulacra of Human Behavior
- Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Generative AI and agent-use policy

Use of AI tools for coding, debugging, research, testing, and documentation is expected and encouraged. Students do not need to disclose individual prompts or identify every AI coding assistant they used.

Each graded submission includes a **verification and reproducibility note** of no more than three bullets:

1. the model or models used by the submitted agent or its evaluation;
2. how the student tested the submission; and
3. one known failure or limitation, or “none found.”

Students do not list coding assistants, prompts, or every AI interaction. The three bullets may appear at the end of the assignment README or report; no separate document is required.

Students remain responsible for the submitted work and must be able to explain, run, test, and modify it. An assignment may restrict AI use only when the restriction is necessary to compare models or measure the effect of AI assistance.

Do not submit private, regulated, proprietary, or personally identifying data to a model or external service. Follow USC’s current data guidance and use institutionally approved tools where required.

Submission, late work, and reproducibility

All graded work is submitted through the course learning-management system. Discussion platforms are not submission channels.

For each programming assignment, each day late incurs a 15% penalty, capped at three days. This policy does not apply to the recorded video presentation deadline or the university-scheduled final submission. Documented emergencies, religious observances, and approved accommodations are handled under current USC policy.

A submission is complete only when a fresh copy of its repository can run the documented tests and evaluation command. The submission must not depend on secret keys, files that were not submitted, or manual steps that were not documented.

Grading scale

Grade	Range
A	95–100
A–	90–94
B+	87–89
B	83–86
B–	80–82
C+	77–79
C	73–76
C–	70–72

Grade	Range
D+	67–69
D	63–66
D−	60–62
F	59 and below